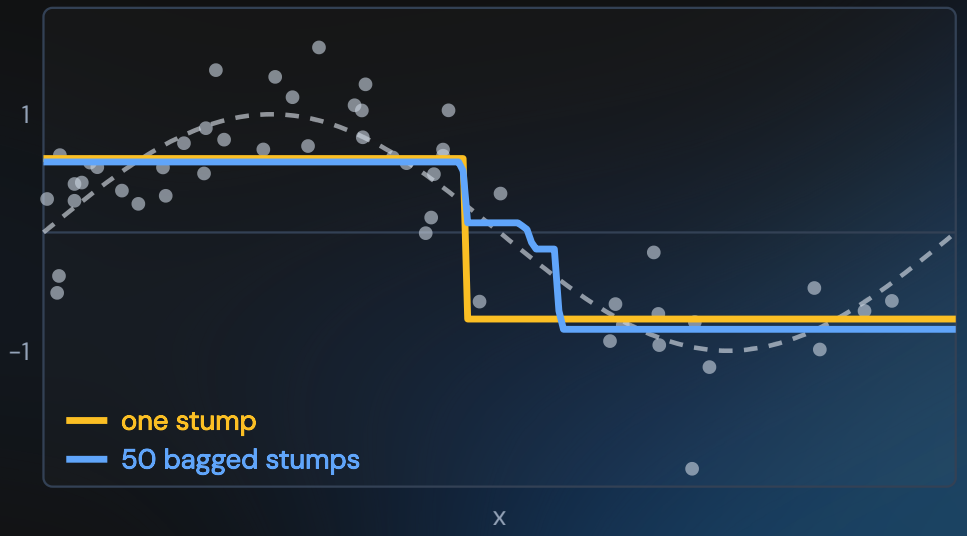
Boosting: AdaBoost and Gradient Boosting

Build strength by learning from mistakes

Ensemble Methods · Foundations of Machine Learning

Where We Left Off: Averaging Cannot Fix Bias

Same 50 points as the bagging deck

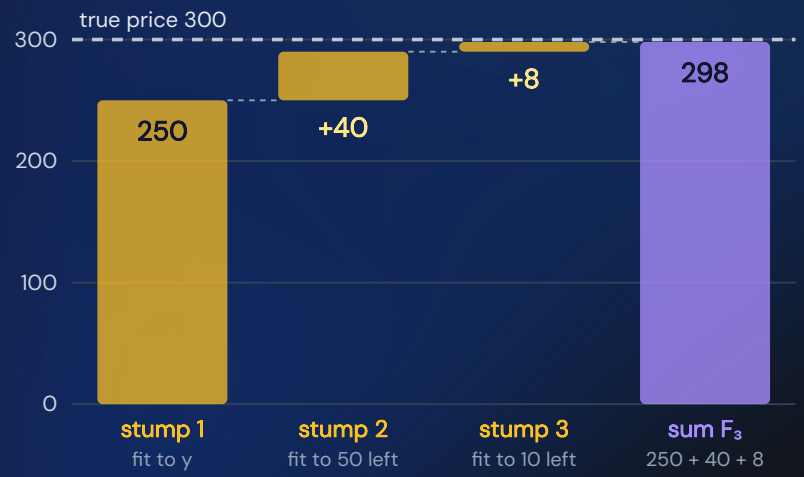


A stump is too simple — One split, two flat pieces. It can't bend to follow a sine wave.

Bagging smooths it, but it still misses — Over 200 datasets: bias² 0.074 → 0.071, variance 0.037 → 0.020.

Today's question: how do we combine weak learners so that bias goes down?

Add, Don’t Average



Each member fits what's left over — Stump 2 never sees the price. It sees the leftover 50.

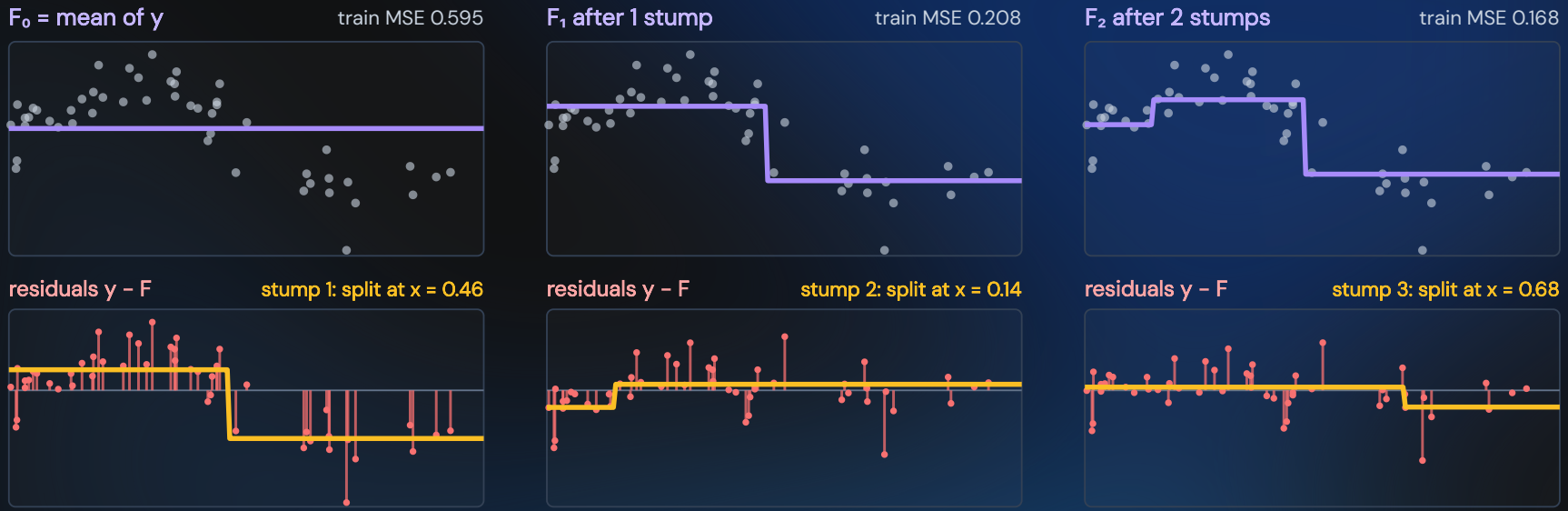
Members are summed — The prediction is the running total.

Not identically trained — Each member depends on the ones before, so the averaging argument no longer applies, and bias can fall.

$$F_M(x) = h_1(x) + h_2(x) + \cdots + h_M(x)$$

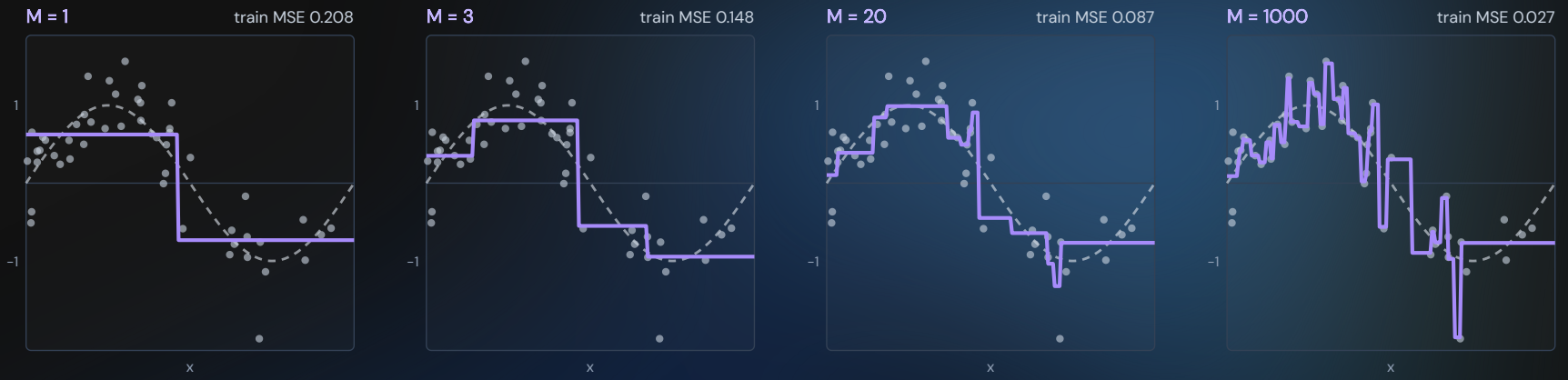
Fit the Residuals, Round by Round

Start from the average of y. Each round, fit a stump to the residuals, then add it to the fit.



Top: the fit so far F. Bottom: what it still gets wrong, y − F, and the stump fit to it. Adding that stump gives the next panel's F.

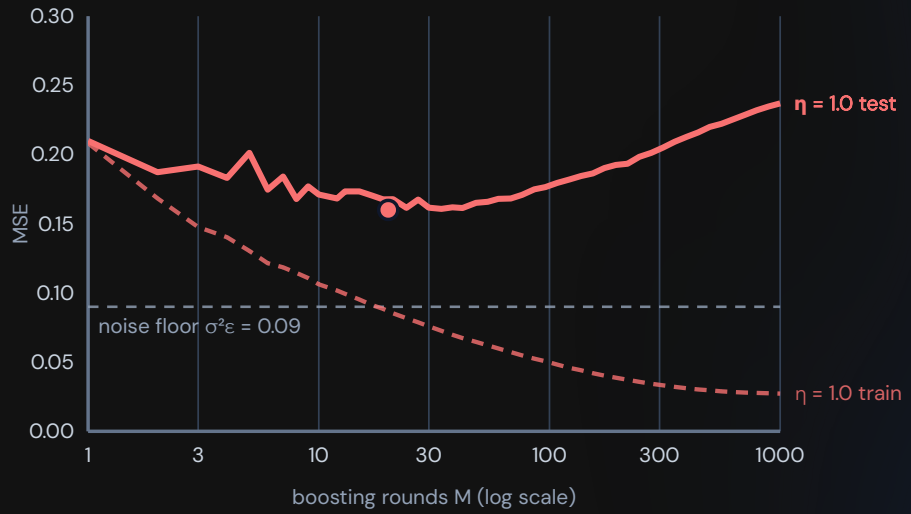
The Ensemble Takes Shape



Bias falls round by round — By 20 stumps the step function traces the sine wave.

Then it keeps going — By 1000 stumps it is fitting the noise. Nothing in the loop says stop.

More Rounds Can Hurt

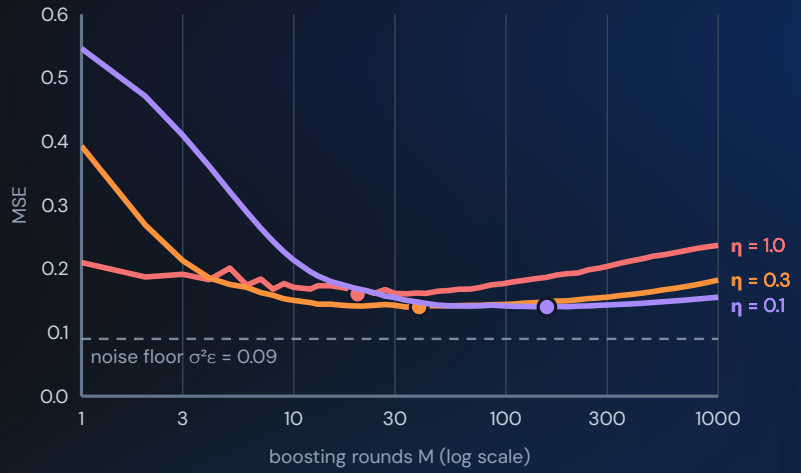


Training error only falls — Every round fits more of what's left, signal or noise.

Test error turns around — Lowest at M = 20 (0.160), then up to 0.237 at M = 1000.

Unlike bagging, the number of members is a knob that overfits. Choose it on held-out data.

Shrinkage: Take Smaller Steps



$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x), \qquad 0 < \eta \leq 1$$

Add only part of each stump — Later rounds get a chance to correct earlier ones.

Slower, but safer — η = 0.1 needs 156 rounds instead of 20, and rises only to 0.156 by M = 1000.

Tune η and M together — Smaller η, more rounds.

Gradient Boosting in a Few Lines

```
eta = 0.1                           # learning rate
F0 = y.mean()                       # start from the average
F = np.full(len(y), F0)
trees = []
for m in range(100):
    residual = y - F                # what is still wrong
    tree = DecisionTreeRegressor(max_depth=1, min_samples_leaf=5)
    tree.fit(X, residual)
    F += eta * tree.predict(X)      # small step
    trees.append(tree)

def predict(X_new):
    return F0 + eta * sum(t.predict(X_new) for t in trees)
```

Keep every tree — The model is F_0 plus all M scaled trees, not the last one.

The same model in scikit-learn — `GradientBoostingRegressor` with the same η, rounds, depth and leaf size gives identical predictions.

Sequential — Tree m needs F from tree m - 1, so rounds can't run in parallel.

Why Is It Called Gradient Boosting?

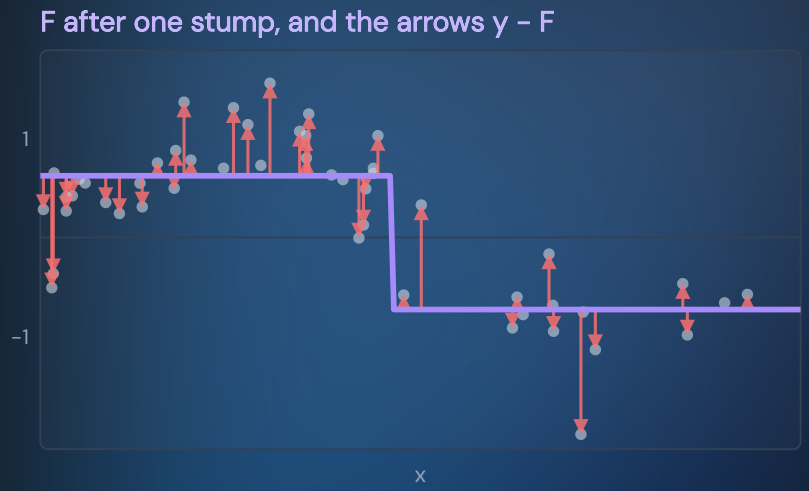
$$\text{Gradient descent:} \quad \theta \leftarrow \theta - \eta \nabla_\theta L$$

$$\text{Boosting:} \quad F(x_i) \leftarrow F(x_i) - \eta \frac{\partial L}{\partial F(x_i)}$$

The arrows are the negative gradient — For ½(y − F)², −∂L/∂F = y − F: the residual.

The tree generalizes the arrows — Arrows exist only at training points. A tree fit to them gives a step at any x.

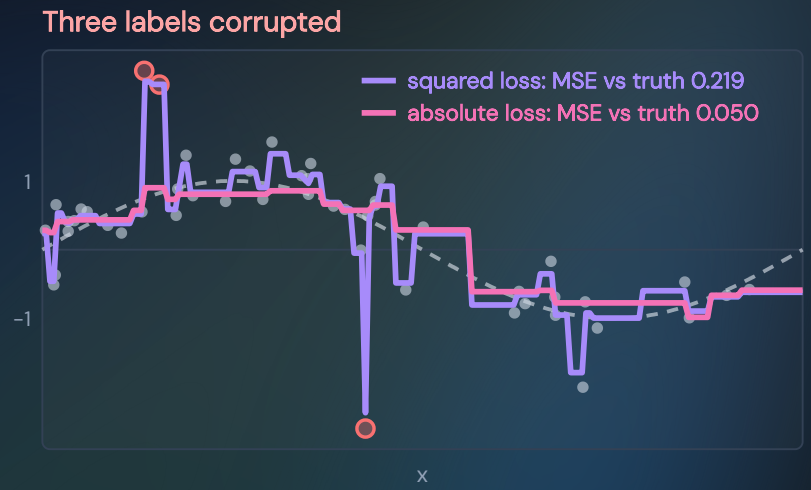
Descent in function space — Each round moves the predictions downhill on the loss.



Choose a Loss, Get a Target

Loss	Each tree is fit to	Typical use
Squared, $\frac{1}{2}(y - F)^2$	$y - F$	regression
Absolute, $\lvert y - F \rvert$	$\text{sign}(y - F)$	regression with outliers
Log loss	$y - p$, where $p = \sigma(F)$	classification
Exponential, e^{-yF}	reweighted labels	AdaBoost

Absolute loss caps each point's pull — A residual of 3 and a residual of 0.3 both send an arrow of size 1.



AdaBoost: Reweight Instead of Refit

1. **Equal weights** on every example, $w_i = 1/n$
2. **Fit a stump** h_t to the weighted data
3. **Weighted error** ε_t = total weight it gets wrong
4. **Its vote** α_t: larger when ε_t is smaller
5. **Reweight:** mistakes up, correct down. Back to 2.

$$\alpha_t = \tfrac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}$$

$$w_i \leftarrow \frac{w_i \, e^{-\alpha_t y_i h_t(x_i)}}{Z_t}$$

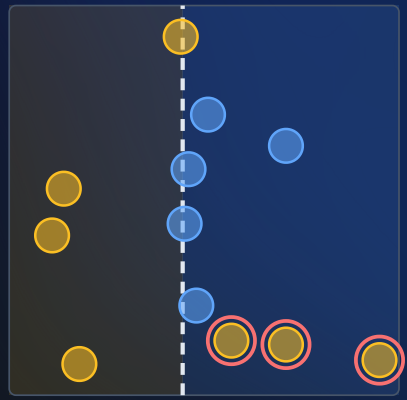
$$H(x) = \operatorname{sign}\left(\sum_t \alpha_t h_t(x) \right)$$

Labels and stump outputs are ±1, so $y_i h_t(x_i) = +1$ when correct and −1 when wrong. Z_t rescales the weights to sum to 1.

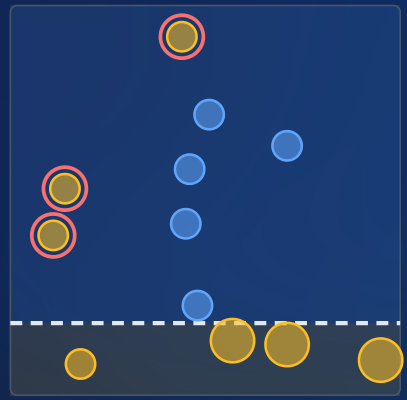
AdaBoost, Round by Round

12 points, two classes. Dot area = example weight. Red ring = this round's stump gets it wrong.

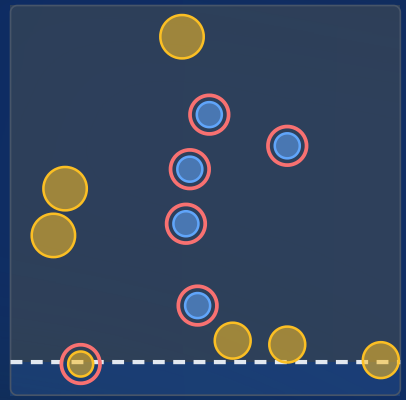
Round 1
$\varepsilon = 0.250$ $\alpha = 0.55$



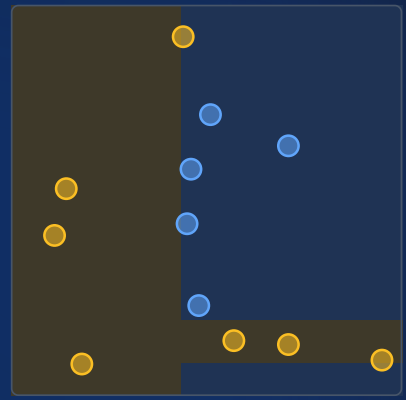
Round 2
$\varepsilon = 0.167$ $\alpha = 0.80$



Round 3
$\varepsilon = 0.200$ $\alpha = 0.69$



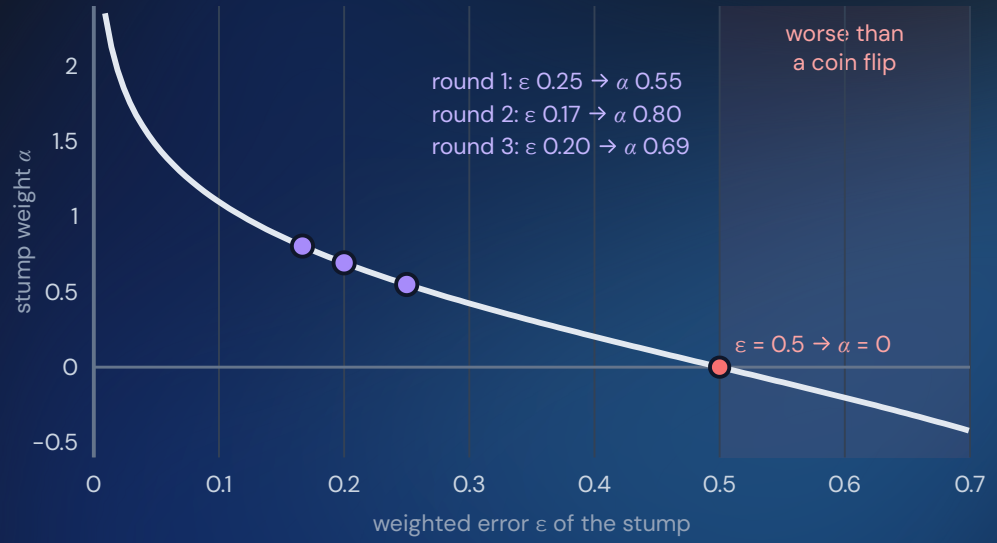
Combined vote
$0.55 \cdot h_1 + 0.80 \cdot h_2 + 0.69 \cdot h_3$



Mistakes grow — After round 1, its 3 mistakes weigh 1/6 each and the rest 1/18. The mistakes now hold half the weight.

No stump is right, the vote is — Each stump misses 3 to 6 points. The weighted vote classifies all 12 correctly.

How Much Say Does a Stump Get?



Accurate stumps vote louder — ε = 0.01 gives α = 2.30. The example's stumps get 0.55 to 0.80.

A coin flip gets no vote — ε = 0.5 gives α = 0. The weak learner must beat chance, as Why Ensembles Work showed for voting.

α also sets the reweighting — Round 1: α = 0.55, so mistakes × $e^{0.55}$ = 1.73 and correct × 0.58, before rescaling.

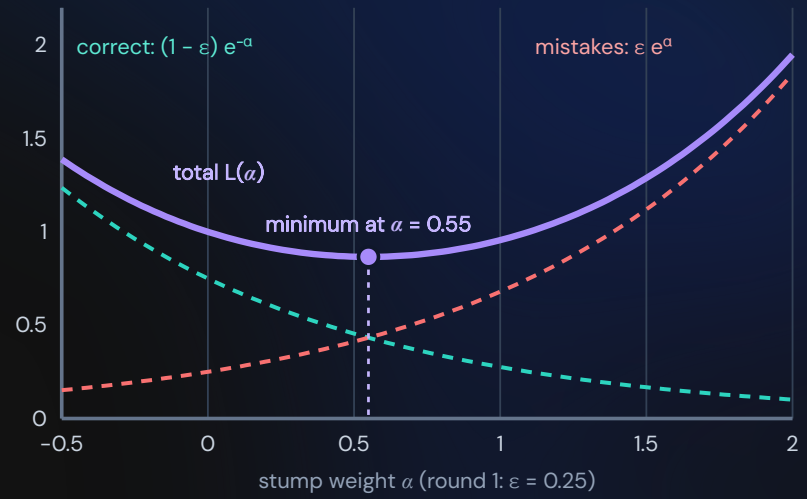
Where α Comes From

Loss of one round — Weighted exponential loss after adding $\alpha \cdot h_t$. Correct examples contribute $e^{-\alpha}$, mistakes e^{α}.

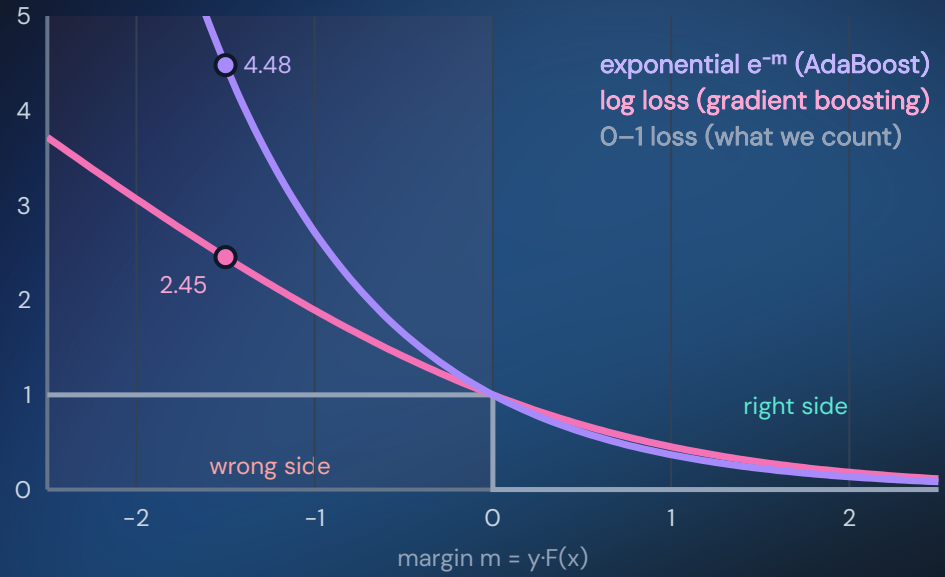
$$L(\alpha) = (1 - \epsilon_t)\, e^{-\alpha} + \epsilon_t\, e^{\alpha}$$

Set the derivative to zero — $-(1-\varepsilon)e^{-\alpha} + \varepsilon e^{\alpha} = 0$, so $e^{2\alpha} = (1-\varepsilon)/\varepsilon$.

$$\alpha_t = \tfrac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}$$



AdaBoost as Gradient Boosting



The weights are the gradient — $w_i \propto e^{-y_i F(x_i)}$, the size of the loss's slope at point i. Reweighting is the gradient step.

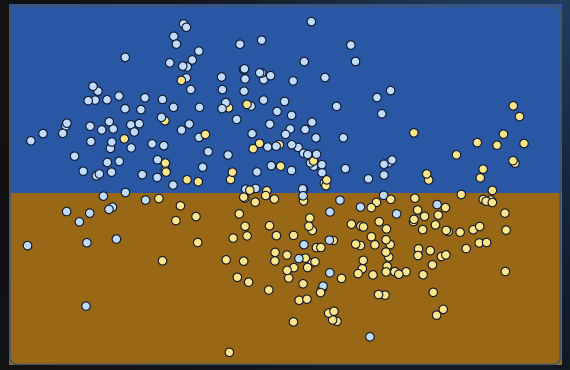
Exponential loss punishes hard — At margin -1.5: 4.48, against 2.45 for log loss (base 2). A mislabeled point can grab most of the weight.

Log loss grows linearly — More forgiving of label noise, and gives probabilities.

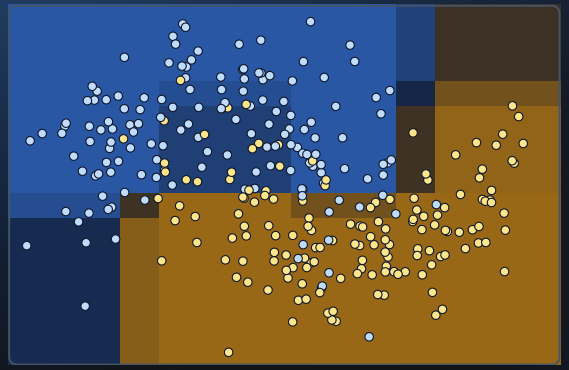
AdaBoost on Two Moons

The same 300 training points as the bagging deck. AdaBoost with stumps.

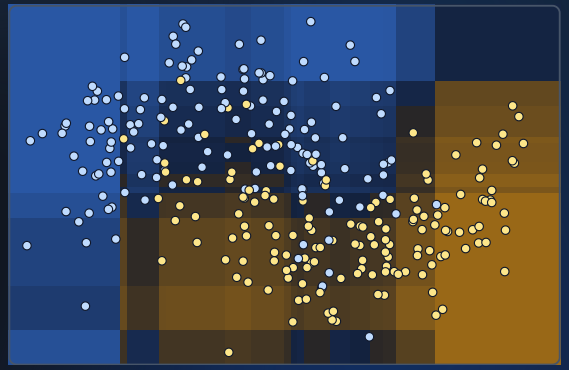
1 stump — test accuracy 80.8%



10 stumps — test accuracy 88.4%



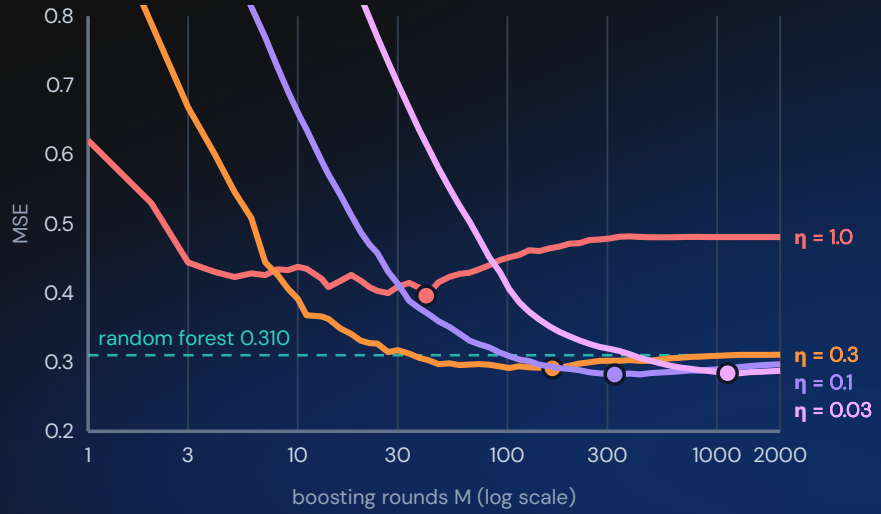
400 stumps — test accuracy 89.1%



Stumps add up to curves — Each stump is one straight cut. Hundreds of them make a staircase that bends.

Bagging deck, same data: one deep tree 83.4%, 300 bagged trees 88.3%. Boosted stumps: 89.1%.

Learning Rate and Rounds on Real Data

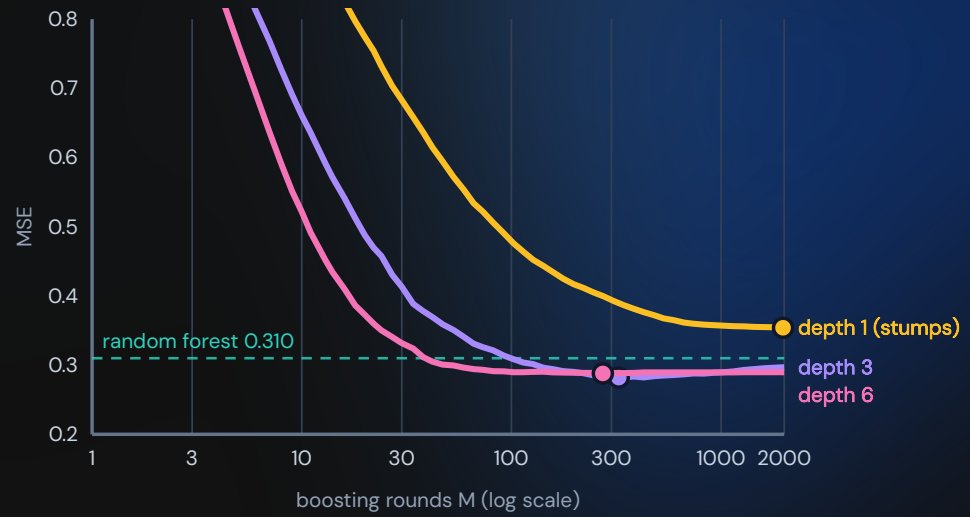


η = 1 overfits fast — Best 0.396 at 41 rounds, 0.481 by 2000. Worse than the random forest throughout.

η = 0.1 gives the lowest error — 0.282 at 325 rounds, and the curve stays low.

Smaller still costs time — η = 0.03 needs 1,123 rounds to reach 0.284.

Tree Depth Controls Interactions



Stumps build an additive model — Each stump uses one feature, so the sum is $f_1(x_1) + f_2(x_2) + \cdots$ with no interactions. Best 0.354.

Depth d allows d-way interactions — Depth 3 reaches 0.282, depth 6 reaches 0.288.

Still weak learners — Shallow, high-bias trees. Boosting supplies the rest.

Early Stopping in Practice

```
from sklearn.ensemble import HistGradientBoostingRegressor

gb = HistGradientBoostingRegressor(
    learning_rate=0.1,
    max_iter=2000,              # an upper limit on rounds
    early_stopping=True,        # 'auto' is off below 10,000 rows
    validation_fraction=0.1,
    n_iter_no_change=10,
    random_state=0,
).fit(X_train, y_train)

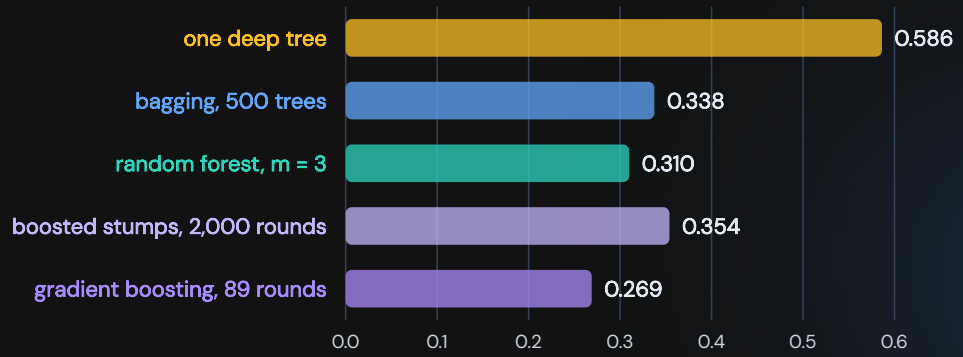
print(gb.n_iter_)               # 89 rounds used
mse = ((gb.predict(X_test) - y_test) ** 2).mean()   # 0.269
```

Let validation pick M — Hold out 10% of the training rows. Stop after 10 rounds with no improvement.

No out-of-bag rows — Every round trains on all the rows, so the validation set has to be carved out.

Histogram-based — Bins each feature into at most 255 values. Much faster on large data.

The Scoreboard: California Housing



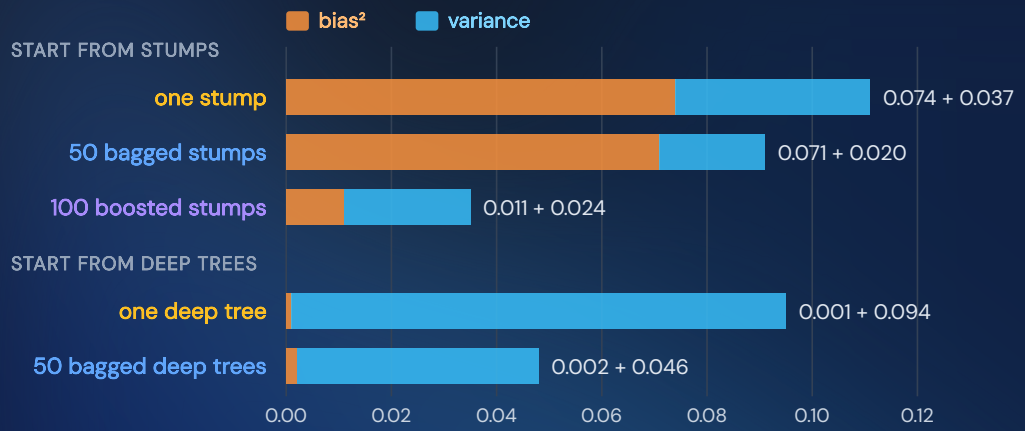
Test MSE on 2,000 held-out districts. Same 2,000 training rows for every model.

Averaging — Bagging and random forests cut a deep tree's error by almost half.

Correcting — Gradient boosting goes further, 0.269. Boosted stumps can't model interactions, 0.354.

The usual pattern on tables: a tuned boosted model wins; a random forest is close with almost no tuning.

Who Reduces What



Sine toy, 200 fresh 50-point datasets. Averaged over x. Noise (0.09) not included.

Bagging cuts variance — Deep trees: 0.094 → 0.046. Bias stays put, for stumps and deep trees alike.

Boosting cuts bias — Stumps: bias² 0.074 → 0.011.

Match the method to the problem.

Name the base learner's weak spot first.

Practical Defaults

Knob	What it controls	Starting point
`learning_rate` η	Step size	0.05–0.1. Smaller needs more rounds.
`max_iter` / `n_estimators`	Rounds M	Set high; let early stopping choose.
`max_depth` / `max_leaf_nodes`	Interactions	Depth 3–8, or about 31 leaves.
`loss`	What counts as wrong	Squared; absolute or Huber† for outliers; log loss for classes.
`subsample` †	Rows per round	0.5–0.8 adds randomness, can cut variance.
`min_samples_leaf` , `l2_regularization` ‡	Leaf size and values	Raise them if the model overfits.

In scikit-learn: `HistGradientBoosting*` for most work. † `GradientBoosting*` only. ‡ `Hist*` only.

Beyond it: XGBoost, LightGBM and CatBoost use the same ideas, with more speed and regularization options.

Ensemble Methods in One View

Average

Bagging: deep trees on bootstrap samples. Cuts variance toward $\rho\sigma^2$.

Decorrelate

Random forests: random features per split. Lowers ρ, so the floor drops.

Correct

Boosting: shallow trees in sequence, each fit to the negative gradient. Cuts bias.

Boosting can overfit — Tune η and M together, and stop early on held-out data.

Nothing beats the noise — Every method here works on bias or variance. None touches σ^2_ε.